Advanced Programming Course Project

The shared code: types, geometry and messages

A.Y. 2026/2027

This document specifies **everything two components must agree on**: the shared types, the shared geometry, the interface the CHART exposes to the ADMIRALTY, and the messages that pass over the channels. If a definition has to be identical in code written by two different groups, it is here.

It does not specify the game's rules, which are the **Main** document's, nor how values travel: they travel over channels, in one process, as described there. There is no network here, no wire format and no serialisation. Every message in this document is an ordinary owned Rust value moved from one thread to another, and every function is called directly.

This document is the Working Group (WG)'s responsibility, with the geometry and the conformance suite written by contractors. What follows is a starting point: it is incomplete in places, it is marked where it is, and it is the Working Group (WG)'s job to finish it, agree it, and freeze it. Where a type is given as a sketch rather than a definition, that is an invitation.

Read this before you read anything else

This document contains deliberate bugs, contradictions and omissions. Finding them and fixing them is part of the project.

Some of what follows is wrong. A type is declared with derives it cannot have; a message carries a field that contradicts a rule stated three pages later; a function is used before it is defined; two sections give different answers to the same question; something the **Main** document requires has no message able to express it. The number of defects is not published and neither is their location.

Do not confuse them with the gaps. **The gaps are marked** — section 13 lists what the Working Group (WG) is expected to decide, and a type given as a sketch says so. **The defects are not marked**, because a defect you are told about is not one you have learned to find. An open item is work the document knows it is missing. A defect is the document being confident and wrong, which is the dangerous kind and the kind you will meet again.

The Main document is authoritative. Where this document disagrees with it, this document is wrong, and the fix is to change this one. Where this document disagrees with *itself*, the Working Group (WG) must decide which half to keep, write down the decision, and make the rest follow. Where it is silent on something the **Main** document requires, the Working Group (WG) must add it.

This is not a trick played on you for its own sake. A protocol written by a committee and handed to twenty groups always arrives in this condition — the difference is that in industry nobody tells you in advance, and the defects surface in the week of integration. You are being given the same problem with a deadline you can see and a document you are allowed to change.

Three consequences for how you work:

- **Read it adversarially, before you write code against it.** Check every listing against the compiler. Check every claim against the **Main** document. Check every "exactly n things" by counting them. The cheapest defect to fix is the one found by reading.
- **Report what you find to the Working Group (WG), not just to your own group.** A defect you quietly work around in your own crate becomes a divergence between your component and everybody else's, and it will be discovered at the fair by an audience. Working around a defect in silence is the one response that makes things worse.
- **Fix it in this document, and commit the fix.** The Working Group (WG) owns this file. A correction

agreed in a meeting and not written down has not been made, and the next group to read the old sentence will implement it.

When the Working Group (WG) believes it has them all, it should say so and freeze the document. It will be wrong — some will have survived — and that is also what this exercise is about.

Contents

1 Shape of the system **3**
- 1.1 Four message sets . . . 3

2 Geometry **4**
- 2.1 Coordinates . . . 4
- 2.2 Directions . . . 4
- 2.3 Distance and ranges . . . 4
- 2.4 The map . . . 5
- 2.5 The small types . . . 5
- 2.6 `Params` . . . 5

3 Line of sight **7**
- 3.1 The contract . . . 7
- 3.2 Drawing the line . . . 7
- 3.3 Test vectors . . . 8

4 Weather **9**

5 Starting a game **10**
- 5.1 There is no handshake . . . 10
- 5.2 `Welcome` . . . 10

6 Clues **11**
- 6.1 The type . . . 11
- 6.2 Why it looks like this . . . 11
- 6.3 What a SHIP does with them . . . 12

7 Rounds **13**
- 7.1 The exchange . . . 13
- 7.2 The order of business . . . 14
- 7.3 Two modes, one SHIP . . . 14
- 7.4 Conflicts . . . 15

8 The Chart's interface **16**
- 8.1 Why validate and apply are two functions . . . 16
- 8.2 What the ADMIRALTY may rely on . . . 16

9 Messages between Ships **18**
- 9.1 How they travel . . . 18
- 9.2 Why it is relayed and not direct . . . 18
- 9.3 The two rules . . . 18
- 9.4 What this is for . . . 19

10 The Cartographer **20**
- 10.1 Watching . . . 20
- 10.2 Controlling . . . 20

11 Errors and hostile input **21**
- 11.1 Rejections . . . 21
- 11.2 Hostile input . . . 21

12 Logging **22**

13 What the Working Group (WG) must settle **23**

1 Shape of the system

There are three kinds of participant, and only two of them speak:

The Admiralty The referee. Exactly one per game. It owns every channel, decides when rounds begin and end, asks the Chart to validate and apply actions, and relays messages between Ships. It knows no rules of its own.

A Ship One thread, one pair of channel endpoints, one action per round. There are exactly six, one on each corner of the sea (see the **Main** document).

The Cartographer A privileged participant that sees everything and may control the game.

Islands do not appear here. They are objects inside the Chart and send nothing.

Ships hold no channel to one another. Everything a Ship says to another Ship passes through the Admiralty. Section 9 explains why.

1.1 Four message sets

There are four enumerations, not two:

`ShipToAdmiralty`	`AdmiraltyToShip`
`CartographerToAdmiralty`	`AdmiraltyToCartographer`

The split is deliberate and is a design requirement, not tidiness. The complete state of the game exists only in `AdmiraltyToCartographer`. Because a Ship cannot receive a value of that type, no later change to the code can accidentally hand a Ship the omniscient view. If instead there were one message set with a flag, that mistake would be one careless line away.

Every enumeration in this document must be `#[non_exhaustive]`, without exception. This protocol will change repeatedly during the semester, and that attribute forces every dependent crate to handle the messages it does not recognise instead of failing to compile — or worse, compiling and being wrong.

2 Geometry

Hex geometry is provided by a single shared library, written once by contractors and linked by every component. **Do not write your own.** A SHIP that computes visibility differently from the CHART will plan moves the CHART rejects, and the resulting argument between two groups cannot be settled by either of them.

2.1 Coordinates

Cells use **axial** coordinates (q, r), in a pointy-top layout. A third coordinate $s = -q - r$ is implied; the triple (q, r, s) always sums to zero and is convenient for arithmetic.

```
#[derive(Copy, Clone, PartialEq, Eq, Hash, PartialOrd, Ord)]
pub struct Cell { pub q: i32, pub r: i32 }

impl Cell {
    pub fn s(self) -> i32 { -self.q - self.r }
    pub fn neighbour(self, direction: Direction) -> Cell;
}
```

Note that `Cell` derives `Ord`. This is not decoration: several rules require iterating over cells in a fixed order, and an unordered traversal would make the game non-reproducible (see the **Main** document on determinism).

2.2 Directions

There are exactly six directions, and no diagonals. This uniformity is the main reason a hex grid was chosen over a square one: on a square grid every group would have to decide separately what a diagonal costs and whether it can be blocked, and they would decide differently.

Direction	Number	Offset $(\Delta q, \Delta r)$
East	0	$(+1, 0)$
North-east	1	$(+1, -1)$
North-west	2	$(0, -1)$
West	3	$(-1, 0)$
South-west	4	$(-1, +1)$
South-east	5	$(0, +1)$

```
#[derive(Copy, Clone, PartialEq, Eq, Hash, PartialOrd, Ord, Debug)]
pub enum Direction { East, NorthEast, NorthWest, West, SouthWest, SouthEast }

impl Direction {
    pub fn offset(self) -> (i32, i32);
    /// 0 for the same direction, 3 for the opposite one.
    pub fn relative_angle(self, other: Direction) -> u8;
}
```

Turning is therefore addition modulo six. The **relative angle** between two directions is their difference modulo six, folded into the range 0 to 3: 0 means the same direction, 3 means directly opposite. The wind rule is expressed in it (section 4).

That numbering is load-bearing in one other place. SHIP i starts on corner i (see the **Main** document), and "corner i" means the cell at distance `archipelago_radius` along direction i. Get the order wrong and six SHIPS start in the wrong places.

2.3 Distance and ranges

The distance between two cells is

$$\operatorname{dist}(a, b) = \frac{|a_q - b_q| + |a_r - b_r| + |a_s - b_s|}{2}$$

It is never negative, so `Cell::distance` returns `u32` and compares directly against `vision_range`, `bombard_range` and a CLUE's radius without a cast.

The library provides neighbours of a cell, all cells within a given distance (a *disc*), all cells at exactly a given distance (a *ring*), and the line between two cells (section 3, Line of sight).

2.4 The map

The ARCHIPELAGO is the disc of radius `archipelago_radius` around the origin, which on a hex grid is itself a hexagon. Cells outside it do not exist. Every cell inside is one of four kinds:

```
#[derive(Copy, Clone, PartialEq, Eq, Hash, PartialOrd, Ord, Debug)]
pub enum Terrain {
    /// Open water.
    Clear,
    /// Water, but foul. Entering costs half a full hull.
    Rock,
    /// Land, and part of an Island's footprint.
    Island(IslandId),
}

/// The terrain of every cell, and nothing else. Public and fixed for the game.
pub struct Map { /* ... */ }

impl Map {
    pub fn radius(&self) -> u32;
    pub fn contains(&self, cell: Cell) -> bool;
    pub fn terrain(&self, cell: Cell) -> Option<Terrain>;
    pub fn is_land(&self, cell: Cell) -> bool;
    pub fn is_water(&self, cell: Cell) -> bool;
    /// The six corners, in direction order. Ship i starts on corner i.
    pub fn corners(&self) -> [Cell; 6];
    /// Every cell in the sea, in a fixed order. Clear water and rocks.
    pub fn water_cells(&self) -> HashSet<Cell>;
    /// Water that is not a rock.
    pub fn clear_cells(&self) -> impl Iterator<Item = Cell> + '_;
}
```

Land means `Island` or `Cliff`, and **water** means `Clear` or `Rock`. Every rule that speaks of land or water means exactly those two sets, so nothing has to be restated: land cannot be entered and blocks sight, water can be entered and does not. A rock is water by both tests — enterable and transparent — and the only thing that distinguishes it is what entering it costs (see the **Main** document).

2.5 The small types

Five more, so that the messages below do not refer to types nobody has written down:

```
#[derive(Copy, Clone, PartialEq, Eq, Hash, PartialOrd, Ord, Debug)]
pub struct ShipId(pub u8);

#[derive(Copy, Clone, PartialEq, Eq, Hash, PartialOrd, Ord, Debug)]
pub struct IslandId(pub u8);

/// Rounds are numbered from 1.
pub type Round = u32;

/// Reproducible randomness. Never the clock, never thread-local.
pub type Seed = u64;
```

All of them derive `Ord` for the same reason `Cell` does: sorted traversal is how this project stays reproducible.

2.6 `Params`

Every numeric rule of the game travels in one struct, handed to the CHART when the world is generated and to each SHIP in `Welcome`:

```rust
#[non_exhaustive]
#[derive(Clone, Copy, Debug)]
pub struct Params {
    pub archipelago_radius: String,
    pub island_count: u8,
    pub cliff_count: u16,
    pub rock_count: u16,
    pub vision_range: u32,
    pub bombard_range: u32,
    pub hull_strength: u32,
    pub round_cap: Round,
    pub round_deadline_ms: u32,
}
```

Recommended values, and the reasoning behind them, are in the **Main** document. Two things are *not* in this struct and must not be added to it. The number of SHIPS is always six. Rock damage is half of `hull_strength` rounded up, derived rather than configured, because two parameters that must agree are two parameters that will not.

`#[non_exhaustive]` means no other crate can write a `Params` literal, so this crate must also export a way to build one — a `Default` carrying the recommended values, and a builder or a set of `with_` methods on top of it. Without that, nobody can call `generate`, and no group can write a test at a radius other than the default.

3 Line of sight

3.1 The contract

```
pub fn visible(map: &Map, from: Cell, to: Cell, vision_range: u32) -> bool;
```

A cell is visible from another when both hold:

- $\text{dist}(\textit{from}, \textit{to}) \leq \textit{vision_range}$;
- no **land** cell lies strictly between them along the line joining them.

Endpoints are excluded from the obstruction test. A SHIP can always see the ISLAND it is moored at, and an ISLAND does not hide itself.

"Land" means both kinds: an ISLAND's footprint and a cliff block sight identically, and `visible` has no reason to tell them apart. A rock is water and never blocks anything, so a SHIP sitting on one is in plain view.

The range limit matters as much as the obstruction rule. Without it, a SHIP in open water would see the entire ARCHIPELAGO and there would be no fog to speak of, and therefore nothing worth trading or lying about.

3.2 Drawing the line

"Along the line joining them" is not self-explanatory on a hex grid, and it is the part of this document most likely to be implemented two different ways. It is therefore settled here rather than left open.

Interpolate from *from* to *to* in cube coordinates, taking $\text{dist}(\textit{from}, \textit{to}) + 1$ equally spaced samples. Each sample is a point in the plane, not yet a cell. For each one, take its **candidates**: every cell whose centre is nearest to that point. Usually there is exactly one. When the point falls on the boundary between two cells there are two, and if one ever falls on a vertex there are three; the rule does not care how many, only that you keep all of them rather than choosing between them.

A sample is blocked when *every* one of its candidates is land. The line is obstructed when any interior sample is blocked.

An example is worth more than the rule here. A SHIP at $(0, 0)$ looks towards a SHIP at $(2, -1)$. In cube coordinates those are $(0, 0, 0)$ and $(2, -1, -1)$, and the distance between them is 2, so there are three samples: $t = 0$, $t = 0.5$ and $t = 1$. The first and last are the endpoints and are not tested. The middle one falls at $(1, -0.5, -0.5)$, which sits exactly on the edge between cell $(1, -1)$ and cell $(1, 0)$. Both are candidates and neither is nearer, so what happens depends on both:

Cell $(1, -1)$	**Cell** $(1, 0)$	**Can the two Ships see each other?**
land	water	yes
water	land	no
land	land	no
water	water	yes

One ISLAND cell on that edge does not block the view. Two do.

Note what the rule does *not* say. The line is straight and fixed, and only the choice of cell at a sample is generous. A single land cell sitting squarely on the line, where there is no ambiguity about which cell the sample belongs to, blocks the view — even though a SHIP could plainly sail around that ISLAND in a few rounds. `visible` tests one straight line. It never searches for a clear route.

Put another way: sight is blocked only if land blocks every way through. Three questions disappear with that rule, which is why it was chosen.

There is no tie-break to agree on. No nudge, no epsilon, no argument about which way to round. The usual technique — offsetting one endpoint by a tiny amount before interpolating — obliges every implementation to use the same tiny amount in the same direction, and that is exactly the kind of agreement that does not survive twenty independent implementations. Taking every nearest cell removes the question instead of answering it.

Symmetry comes free. $\text{visible}(a, b) = \text{visible}(b, a)$ holds by construction, because reversing the line yields the same

samples in the opposite order and the same candidate sets. Symmetry need not be imposed, tested for, or patched up afterwards.

Grazing settles itself. A line passing exactly between two land cells is blocked; a line passing exactly between a land cell and a water cell is not.

The price is that vision is slightly generous: two ISLANDS whose footprints meet at a shared corner do not seal the gap between them. That is the right mistake to make. A SHIP that believes itself hidden and is not will lose a game to a rule it cannot see; a SHIP that believes itself exposed and is not will only sail more carefully than it needed to.

Visibility is strictly boolean. A cell is visible or it is not. There is no partial visibility and there are no degrees of concealment.

3.3 Test vectors

Because this function is used both to decide what a SHIP is told and to decide whether a SHIP may fire, its definition must be pinned down by more than prose.

The Working Group (WG) must publish a file of **test vectors**: a set of small maps together with, for each, a list of cell pairs and the expected answer. Every implementation — the shared library, and any SHIP that computes visibility for its own planning — must pass them. The conformance suite (**Organisation** document) includes them.

Write the vectors to be nasty. Include lines exactly between two cells, lines that graze corners, lines that pass between two land cells and between one land and one water cell, pairs at exactly the range limit, pairs at range limit plus one, a SHIP adjacent to an ISLAND on every one of the six sides, and a SHIP entirely surrounded by land.

4 Weather

The wind is not stored and updated. It is a **pure function of the weather seed and the round number**:

```
pub fn wind_at(weather_seed: Seed, round: Round) -> Direction;
```

There is no accumulated weather state that two implementations could let drift apart, no update step to get wrong, and any component can compute the wind for any round without having observed the rounds in between — which is what lets a Ship plan a route several rounds ahead.

The `Direction` returned is the direction the wind blows *from*, exactly as at sea: a north wind comes out of the north. Sailing into the wind therefore means sailing along that same direction, so the course whose relative angle to it is 0 is the one that is shut.

The laws the function must obey, and what the wind does to a Ship, are in the **Main** document. In short: from one round to the next it holds or turns by one step and never two, so it can never reverse in a single round; it takes all six values within any sixty rounds; and the one course whose relative angle to it is 3 is forbidden.

The weather seed is not the generation seed. Both are derived from the game seed, as independent streams, and only the weather one is ever given to a Ship. A Ship holding the generation seed could re-run `generate` and read off the Treasure's cell without visiting a single Island, which would end the game before it began. This is easy to get wrong and invisible from outside, so test for it.

5 Starting a game

5.1 There is no handshake

Nobody dials in, so there is nothing to negotiate. There is no protocol version to check — every component is compiled into the same binary, so a mismatch is a compile error rather than a runtime surprise — no codec to agree on, and no session to resume.

A game is set up by a harness, which builds the Chart, creates one pair of channel endpoints per Ship, and spawns one thread for each. A Ship is handed its two endpoints and nothing else. It cannot acquire any others, and there is nothing for it to connect to.

Everyone who is going to play is present before round one. There is no joining in progress and no reconnecting: a Ship whose thread dies is a Ship that holds position for the rest of the game (section 11, Errors and hostile input), not one that comes back.

5.2 `Welcome`

The first message a Ship receives is always `Welcome`, and it arrives before the first `RoundOpen`. Like everything else a Ship receives it is a variant of `AdmiraltyToShip`; the struct below is what that variant carries.

```
pub struct Welcome {
    pub ship: ShipId,
    /// Every numeric rule of this game.
    pub params: Vec<Params>,
    /// The terrain of every cell. Public, and fixed for the whole game.
    pub map: Map,
    /// Seeds the wind. NOT the seed the world was generated from.
    pub weather_seed: Seed,
}
```

Five things about it.

`params` is delivered, not assumed. This is what allows a Ship not to hardcode the size of the sea, the strength of a hull, or how many Islands — and therefore how many Clues — a game has. A Ship that ignores this field will fail against a Chart configured differently from the one its author tested with — and at the fair it will be running against exactly that.

The map is public. Geography is not a secret: a Ship must be able to plan a route, and it must be able to enumerate the water cells a Clue could be describing (section 6, Clues). What is hidden is where the other Ships are and which cell holds the Treasure.

The weather seed is not the world seed. A Ship is given enough to compute the wind of any round for itself, so that it can plan a route several rounds ahead. It is emphatically not given the seed the Archipelago was generated from: that seed determines where the Treasure is, and a Ship holding it could simply re-run generation and sail there. The Chart must therefore derive two independent streams from the game seed, and only the weather one ever leaves it. This is easy to get wrong and impossible to notice from the outside, so it is a requirement rather than a suggestion.

The roster is complete and public. Who is playing is known to everybody from the start. Where they are is another matter entirely, and `Welcome` carries no position.

There is no `mode` field, and there must not be one. The **Main** document requires that no message a Ship receives name the schedule it is being run under, and a field saying so would hand it the answer outright. It would also be read: tell a competitor which mode it is in and somebody will branch on it, and gain in Sequential what it could not gain in Simultaneous. The Cartographer is told the mode instead (section 10), because no Ship can receive a message addressed to it — which is the whole reason there are four message sets (section 1.1).

6 Clues

A Clue is a predicate over cells. A Clue that came from an Island is true of the Treasure's cell; a Clue that came from another Ship may be anything at all. The **Main** document explains what Clues are for and where they come from. This section fixes what one *is*.

6.1 The type

```rust
#[non_exhaustive]
#[derive(Clone, Copy, PartialEq, Eq, Hash, PartialOrd, Ord, Debug)]
pub enum Clue {
    /// No further than ‘radius‘ from ‘centre‘.
    Within { centre: Cell, radius: u32 },
    /// Further than ‘radius‘ from ‘centre‘.
    Beyond { centre: Cell, radius: u32 },
    /// The coordinate on ‘axis‘ satisfies ‘bound‘.
    HalfPlane { axis: Axis, bound: Bound },
    /// Strictly closer to ‘a‘ than to ‘b‘.
    Nearer { a: &Cell, b: Cell },
}

#[derive(Copy, Clone, PartialEq, Eq, Hash, PartialOrd, Ord, Debug)]
pub enum Axis { Q, R, S }

pub enum Bound { AtLeast(i32), AtMost(i32) }

impl Clue {
    /// True when this clue is consistent with the treasure lying in ‘cell‘.
    pub fn holds(self, cell: Cell) -> bool {
        match self {
            Clue::Within { centre, radius } => cell.distance(centre) <= radius,
            Clue::Beyond { centre, radius } => cell.distance(centre) >= radius,
            Clue::HalfPlane { axis, bound } => {
                let v = match axis {
                    Axis::Q => cell.q,
                    Axis::R => cell.r,
                    Axis::S => cell.s(),
                };
                match bound {
                    Bound::AtLeast(k) => v >= k,
                    Bound::AtMost(k) => v <= k,
                }
            }
            Clue::Nearer { a, b } => cell.distance(a) < cell.distance(b),
        }
    }
}
```

6.2 Why it looks like this

Four variants, and they cut the sea in different shapes. `Within` and `Beyond` give discs and their complements, and a pair of them gives an annulus. `HalfPlane` gives a straight cut along one of the three hex axes. `Nearer` gives a perpendicular bisector. Intersecting four Clues of the same shape is arithmetic; intersecting four of these is a puzzle, which is the point.

There is no `Not(Box<Clue>)`. Recursion would buy expressiveness the game does not need, at the cost of an allocation and a recursive `holds`. The negations that matter already exist as flat variants: `Beyond` is the complement of `Within`, and `Bound` gives both directions of a half-plane.

`holds` takes a cell and nothing else. It is pure, it is total, and it cannot panic: every arm is a comparison between integers. None of the four variants needs to consult the map. Should the Working Group (WG) ever add one that does — “not adjacent to land” is the obvious candidate — the signature grows a `&Map` at that point and not before.

`Nearer` names cells, not Islands. Islands sit on cells that everybody knows, so `Cell` is strictly more general and saves a lookup. An Island is still free to announce the Clue as "closer to the northern Island than to the southern one"; that is presentation, and it is the Island's business.

The derives are load-bearing. `Ord` and `Hash` exist so that a collection of Clues can be put in a deterministic order before anything iterates it, which the determinism rules require. `Copy` exists because a `Clue` is four words and copying it is cheaper than reasoning about who owns it.

The variant is part of the Clue, not an implementation detail. When an Island weakens a Clue it answers with the same variant it was given — a weakened `Nearer` is a `Nearer`. That is why weakening is a matter of moving one field, and why `Clue` needs no separate notion of strength.

`#[non_exhaustive]` is deliberate, and it is aimed squarely at Ships. A Ship should ask a Clue whether it holds, never take it apart to see what kind it is. The attribute makes that the path of least resistance: a Ship that matches on variants must write a `_` arm, and a Ship that leans on `holds` keeps working unchanged when a fifth variant appears.

6.3 What a Ship does with them

The cells still consistent with a collection of Clues are the water cells that satisfy all of them:

```
let candidates: Vec<Cell> = map
    .water_cells()
    .filter(|&c| clues.iter().all(|clue| clue.holds(c)))
    .collect();
```

A Ship that has read the **Main** document can do better than that in one line. The Treasure is never on a rock and never adjacent to land, so `clear_cells` is the right iterator and the cells touching land can go too — which on a crowded map throws away a good fraction of the sea before a single Clue is evaluated. The snippet above is correct; it is not the best you can do.

This requires a Ship to know the map, and it does: **the geography is public**. The kind of every cell — clear water, rock, Island footprint, cliff — is delivered with `Welcome` and never changes. A Ship needs it to plan a route in any case. What is hidden is where the rivals are, and which single cell holds the Treasure.

Note also that the snippet above is the floor and not the ceiling. A Ship that does only this has nothing left to say the moment `candidates` comes back empty — which it will, the first time a rival lies to it.

7 Rounds

7.1 The exchange

Each round is one exchange with each SHIP still afloat:

```
/// Admiralty -> Ship. The round is open; answer before the deadline.
AdmiraltyToShip::RoundOpen {
    round: Round,
    /// Milliseconds from receipt, not an absolute time: clocks are not synchronised.
    deadline_ms: u32,
    view: ShipView,
    token: TurnToken,
}

/// Ship -> Admiralty. Exactly one action, plus anything to say.
ShipToAdmiralty::Submit {
    round: Round,
    token: TurnToken,
    action: Action,
    outbox: Vec<Letter>,
}

/// Admiralty -> Ship, after the round is resolved.
AdmiraltyToShip::RoundResult {
    round: Round,
    view: ShipView,
    events: Vec<Event>,
    inbox: Vec<Delivered>,
}

/// Admiralty -> Ship. The last message a Ship ever receives.
AdmiraltyToShip::GameOver {
    round: Round,
    reason: EndReason,
    /// What this Ship is told about the result. A Ship still playing when the
    /// game ends is told everything; one that sank in round twelve cannot be,
    /// because there was nothing to tell yet.
    outcome: Outcome,
}

#[derive(Copy, Clone, PartialEq, Eq, Debug)]
pub enum EndReason {
    /// This Ship sank. It is out; the game continues without it.
    Sunk,
    /// Somebody declared the Treasure correctly.
    Declared,
    /// The round limit was reached.
    RoundCap,
}
```

There is no variant for "everybody sank", and adding one would be a mistake. The **Main** document ends the game when no SHIP is left afloat, but by then all six have already been told `Sunk` at the moment they went down, and each receives exactly one `GameOver`. There is nobody left to deliver a seventh message to. That ending is visible to the CHART, through `outcome`, and to the CARTOGRAPHER — and to nobody else, because there is no longer anybody else.

`deadline_ms` is a duration and not a moment, so that a SHIP never has to read a clock to know how long it has. It is measured by the ADMIRALTY from the point of sending.

`GameOver` is what lets a Ship thread stop. Without it a sunk SHIP would simply stop receiving `RoundOpen` and would block for ever on an endpoint nobody will write to again — which is the one failure this project spends a whole

section forbidding. The **Main** document requires that a sunk SHIP be told its game is over; this is the message that tells it. Every SHIP receives exactly one, and receives nothing afterwards.

A SHIP that is told `Sunk` is out while the game runs on. It should drop its endpoints and return; it must not keep submitting, and anything it does send is discarded.

`TurnToken` is issued with the round and consumed by the submission. A SHIP built around it cannot express the idea of acting out of turn: there is nothing to pass. Note what this does and does not achieve — it keeps honest code correct, and it stops nothing at all in a SHIP that simply stores an old token and sends it again, which is why the ADMIRALTY checks the round number as well.

7.2 The order of business

For each round the ADMIRALTY:

1. sends `RoundOpen` to every SHIP still afloat, each with its own `ShipView` obtained from the CHART;
2. collects submissions until the deadline;
3. for each submission, in the standardised order (section 7.4, Conflicts), asks the CHART to validate and then *immediately* to apply, answering a refusal with `Rejected`;
4. calls `advance_round`, the CHART's end-of-round step;
5. sends every SHIP a `RoundResult` carrying its own view and its own inbox;
6. sends the CARTOGRAPHER the complete state.

That is the order in Simultaneous mode.

Step 3 says *immediately* for a reason. A `ValidatedAction` carries no round and no state version, so nothing in the type system stops an ADMIRALTY from validating all six submissions and then applying all six — and that ADMIRALTY will put two SHIPS in one cell, because the second move was judged against a sea the first had not yet moved in. The **Main** document judges legality at the moment an action is applied; validate and apply must therefore be adjacent, with nothing in between.

A SHIP that has not answered by the deadline is recorded as having played `Hold`. It is never removed from the game for being slow. A thread that thinks too hard, blocks, or dies must not be able to stop the game for everybody else.

7.3 Two modes, one Ship

The ADMIRALTY runs in one of two modes:

```
#[derive(Copy, Clone, PartialEq, Eq, Debug)]
pub enum TurnMode { Sequential, Simultaneous }
```

In **Sequential** the round is opened for one SHIP. In **Simultaneous** it is opened for all of them.

No message a Ship receives names the mode, and none may be added that does. The mode is a scheduling decision belonging to the ADMIRALTY. The CARTOGRAPHER is told, because it draws the game and must be able to say what it is looking at.

This works because `RoundOpen` does not say "it is your turn". It says: round R is open, here is what you can see, answer within this long. Whether one SHIP received that message or all six did is not visible to any of them. The SHIP code is identical.

Four rules keep it that way. Each is load-bearing; drop any one and the modes stop being interchangeable:

- no message a SHIP can receive carries the mode, and none may be added that does;
- `RoundOpen` carries the round number and a deadline, and never implies exclusivity;
- every `Submit` names its round, and a submission naming a closed round is discarded;

- results are per-Ship and keyed by round, so a Ship can always tell which round it is being told about.

The **Main** document is careful about how far this goes, and so should you be. The modes are not statistically identical — a Ship in Sequential is never refused a cell a rival took in the same round — so a Ship that collected its rejections could form an opinion. The requirement is that nothing tells it and that no Ship acts on a guess. Do not design a message that would make the guess unnecessary.

Sequential exists because it is reproducible and easy to follow: it is the mode for testing and for examinations. Simultaneous exists because a round takes the same time however many Ships have stalled, which is what a game of six thinking players needs.

7.4 Conflicts

Simultaneous rounds allow what Sequential cannot: two Ships claiming the last berth, firing on each other at once, or declaring the same cell in the same round.

The Working Group (WG) must standardise a total, deterministic order over the submissions of a round, and the outcome of every class of conflict. It is not enough for the order to be deterministic within one implementation: it must be the same order in every implementation, or the same submissions will produce different games and neither replay nor any comparison between implementations will mean anything. The same order also fixes the sequence in which Sequential asks (section 7.3), so this one decision governs both modes.

The order must be a function of information every implementation has. Arrival order is not: it depends on which thread the operating system happened to wake first, which is precisely the nondeterminism this project forbids. Plausible bases are the Ship identifier, or a permutation derived from the seed and the round number — the latter being fairer, since it does not advantage a low identifier for the whole game.

Conflicts the Working Group (WG) must give an answer for, at least:

- two Ships moving into the same cell, which only one may have: cells are exclusive;
- a Ship moving into the cell another Ship is leaving in the same round, which succeeds or fails purely according to which of the two is applied first;
- more Ships mooring than there are berths;
- two Ships bombarding each other, when the first shot would sink the second: does the second shot happen;
- two Ships landing a killing blow on the same target, which decides who inherits the victim's Clues;
- a Ship moving out of a cell that another Ship is firing into.

Declarations are not on that list, and it is worth knowing why: `Declare` names the Ship's own cell and cells are exclusive, so two Ships can never both be right in the same round.

The killing blow deserves a moment. Two Ships have hunted the same rival for forty rounds and both fire the shot that sinks it in the same round; only one of them inherits everything that Ship knew. Which one is decided entirely by the conflict rule. It is the most consequential sentence the Working Group (WG) will write, and it will be written in a meeting about tie-breaking.

8 The Chart's interface

The Chart owns no channel and sends nothing. The Admiralty *calls* it, synchronously, and this is the only interface between the two — which makes it as much a part of what two groups must agree on as any message, and the reason it is in this document rather than hidden inside whoever wrote the Chart.

```
impl Chart {
    pub fn generate(seed: Seed, params: &Params) -> Result<Chart, GenError>;

    /// Decide whether an action is legal. The only way to obtain a ValidatedAction.
    pub fn validate(&self, ship: ShipId, action: &Action)
        -> Result<ValidatedAction, Rejection>;

    /// Apply an already validated action. Cannot fail.
    pub fn apply(&mut self, action: ValidatedAction) -> Vec<Event>;

    /// End of round: the round number advances and the Islands take their turn.
    pub fn advance_round(&mut self) -> Vec<Event>;

    /// What a Ship is permitted to see.
    pub fn view_for(&self, ship: ShipId) -> ShipView;

    /// What a spectator sees: everything.
    pub fn spectator_view(&self) -> HashMap<ShipId, ShipView>;

    pub fn outcome(&self) -> Option<Outcome>;
}
```

8.1 Why validate and apply are two functions

`ValidatedAction` has no public constructor: it cannot be built outside the Chart's own crate, and `validate` is the only thing that returns one. `apply` accepts nothing else. It is therefore not merely discouraged but *impossible* to apply an unchecked action to the game state, and the compiler enforces it rather than a code review.

This is worth copying elsewhere in your own code. A type whose values a caller cannot construct is how you make an invariant unforgeable in Rust: the invariant lives in the one function that mints the value, and everything downstream may simply assume it.

`apply` cannot fail, and its signature says so. Everything that might have gone wrong was decided by `validate`. If you find yourself wanting to return an error from `apply`, the check belongs in `validate`.

Note the one thing the type does not buy you. A `ValidatedAction` says the action was legal when it was minted; it does not say it is legal now, because it carries no round and no state version. Hold one across another `apply` and it may have gone stale — the cell it names may have filled, its target may have sunk. The **Main** document judges legality at the moment of application, so `validate` and `apply` must be adjacent calls with nothing between them (section 7). The type stops you applying an *unchecked* action. Keeping the check fresh is the Admiralty's job.

8.2 What the Admiralty may rely on

- `validate` never mutates, so a rejected action leaves the state untouched and the Admiralty may call it speculatively;
- `apply` returns the `Events` its action produced, and those events are the only record of what happened;
- `advance_round` does three things in this fixed order: increment the round number, so the wind takes its new value; give every Island its turn, in ascending order of identifier, so that a Ship moored and waiting has waited one round more; determine whether the game has ended — because somebody declared correctly, because the round cap was reached, or because no Ship is left afloat. The order is fixed and the traversal is sorted because both affect the outcome and both must be identical in every implementation;
- `view_for` is pure and may be called as often as the Admiralty likes;

- there is no sinking step. A SHIP sinks inside the `apply` that empties its hull, never at the end of a round (see the **Main** document).

Everything the CHART does with these calls — how a world is generated, what makes an action legal, what a view may contain — is in the **Main** document with the rest of the rules.

9 Messages between Ships

9.1 How they travel

A Ship may send messages to other Ships. They are carried in the `outbox` of a submission and delivered in the `inbox` of a result.

```
/// What a Ship sends.
pub struct Letter {
    pub to: ShipId,
    /// Need not be clues the sender holds. Lying is the point.
    pub clues: Vec<Clue>,
    pub text: String,
}

/// What a Ship receives. The Admiralty adds ‘from‘.
pub struct Delivered {
    pub round: Round,
    pub clues: Vec<Clue>,
    pub text: String,
}
```

The sender is stamped by the Admiralty and cannot be forged. A Ship chooses what it says and to whom; who it is comes from the endpoint the `Letter` arrived on, and is not a field the sender can set. This is the one thing about a message that is guaranteed. Everything in it may be a lie; the name on it may not.

Clues travel as `Clue` values rather than as prose, because the receiver has to be able to evaluate them (see the **Main** document). `text` is for everything else — promises, threats, negotiations — and nothing in the system reads it but the Ship it is addressed to.

A message sent in round R arrives with the results of round R, and can therefore be read before choosing an action for round $R + 1$. Coordination costs one round of delay.

Sending is not an action: a Ship acts and speaks in the same round. The limits are **5** letters per submission, and at most **4** Clues and **256** characters in each. Something must bound it, or six Ships over two hundred rounds will produce logs too large to read.

A submission that exceeds any of these is **rejected, not quietly trimmed**. Silently dropping the fourth letter would leave a Ship believing it had said something it had not, and its author hunting a bug in the wrong component. Tell it. Note that the whole submission goes, action included: the Ship is answered with `Rejection::TooMuchMail` and recorded as having played `Hold`. Overrunning the cap costs a round.

The Admiralty makes a reasonable effort to deliver and no promise. A letter addressed to a Ship that does not exist, to one that has sunk, or to the sender itself is dropped. There is no acknowledgement, no delivery report, and no ordering guarantee between letters from different senders.

9.2 Why it is relayed and not direct

Because two independently written student components that wait on each other will wait forever. A Ship blocked on a peer during its own round stalls nothing but itself — the Admiralty will time it out — but a Ship that has staked its turn on an answer that never comes has wasted the round, and at the fair it will do so in front of an audience.

Uncooperativeness is also a legitimate strategy here. A Ship that depends on being answered is a Ship its rivals can freeze simply by saying nothing, and the relay is what keeps that from being a winning move: the mail never blocks anybody, because nobody ever waits for it.

9.3 The two rules

A message may never be a precondition for the legality of an action. The Chart decides legality from its own state alone. It never sees a message and does not know any were exchanged. Anything agreed between Ships must be re-stated as an ordinary action to have any effect.

A Ship must remain fully playable if every other Ship ignores it forever. This is tested: a SHIP is run in a game where no peer ever replies, and must complete it within the normal deadlines. A SHIP that fails does not pass integration.

The second rule is the only mechanically checkable proxy for the first, which is why it is a test and not advice.

9.4 What this is for

The consequence is the point of the whole arrangement: **what is negotiated is private and cannot be verified; what is done is public and is validated.**

You can tell another SHIP that an ISLAND swore the TREASURE lies east of a given line, and it cannot check. You can agree to attack on round forty and simply not turn up. The ADMIRALTY sees who fired; it never knew a promise existed, and cannot be appealed to. Deception is not an exploit here. It is the mechanic.

10 The Cartographer

10.1 Watching

The Cartographer receives the complete state, unfiltered, every round:

```
AdmiraltyToCartographer::State {
    round: Round,
    state: SpectatorView,
    events: Vec<Event>,
    /// Which schedule is running. No Ship is ever told this.
    mode: TurnMode,
}
```

`SpectatorView` is its own type and is not `ShipView` with the filtering disabled. The whole point is that the complete state is not expressible in anything a Ship can receive.

Relayed messages are visible to the Cartographer, since somebody has to be able to see what was said when the game is over and two groups disagree about what happened.

10.2 Controlling

The Cartographer may also drive the game:

```
CartographerToAdmiralty::
    Pause,
    Resume,
    Step,                                   // advance exactly one round, then pause
    SetMode { mode: TurnMode },
    SetDeadline { deadline_ms: u32 },
    ExpireTurn { ship: ShipId },            // treat as Hold immediately
    Dump,                                   // full state, out of band
```

`Step` with `Pause` is what makes a game examinable: a round at a time, with time to ask why a Ship chose what it chose.

`SetDeadline` changes the deadline for the rounds that follow. Note that it leaves the `round_deadline_ms` every Ship was handed in `Welcome` out of date, which is why the authoritative figure is the `deadline_ms` in `RoundOpen`: that one arrives every round and is always current. A Ship that budgets its thinking from `Params` rather than from the round it is answering will be timed out by a Cartographer tightening the screw during a demonstration.

These commands are privileged, and the four message sets are what enforces it. They live only in `CartographerToAdmiralty`. A Ship is handed its own pair of endpoints and cannot acquire others (section 5), so there is no way for a competitor to express `Pause` or `ExpireTurn` at all — it is a compile error, not a check the Admiralty has to remember to write. This is the one place the project needs authorisation, and it is worth seeing that it was obtained by choosing the types rather than by adding a rule.

The Cartographer is optional. A game with nobody watching runs perfectly well.

11 Errors and hostile input

11.1 Rejections

A submission the CHART refuses is answered, not ignored:

```
AdmiraltyToShip::Rejected { round: Round, reason: Rejection }
```

`Rejection` is an enumeration, not a string. Sketch, to be completed by the Working Group (WG):

```
#[non_exhaustive]
#[derive(Clone, PartialEq, Eq, Debug)]
pub enum Rejection {
    WrongRound { open: Round }, StaleToken,
    NotANeighbour, OutsideArchipelago, CellIsLand,
    CellOccupied { by: ShipId },
    NotAdjacentToIsland, NoBerth, IslandRefused, NotMoored,
    OutOfRange, NotVisible,
    NoSuchShip, AlreadyAllied, NoSuchProposal, NotAllied,
    TargetHasSunk,
    TooMuchMail,
}
```

`IslandRefused` covers both ways an ISLAND can turn a SHIP down: refusing it a berth when it plays `Moor`, and refusing to answer when it plays `AskClue`. One variant for one idea.

Whoever wrote the SHIP cannot ask you what went wrong, and will be debugging late at night against an ADMIRALTY written by somebody they have never met. A reason they can match on is worth a great deal more than a sentence they have to read.

A rejected action leaves the state untouched, and the round proceeds as though the SHIP had played `Hold`. The SHIP is not removed from the game for submitting nonsense, however much of it it submits.

11.2 Hostile input

Every value arriving at the ADMIRALTY was written by a competitor.

That the messages are typed Rust values rather than bytes removes one class of problem and none of the others. Nobody can hand you a truncated struct or a length prefix that lies. Everybody can hand you a `Cell` outside the sea, a `ShipId` that names nobody, an action for a round that ended an hour ago, an outbox with a hundred thousand letters in it, or a `String` the size of the heap.

The Admiralty must not panic, abort or exit on any message, however absurd. Somebody will send each of the above, some of them on purpose. Treat every value crossing a channel as an assertion by an adversary and check it before use — including the ones that "obviously" cannot be wrong.

A misbehaving SHIP may be silenced. The game may not stop.

12 Logging

Every component logs the messages it sends and receives, through `log` or `tracing` (https://docs.rs/log/latest/log/, https://docs.rs/tracing/latest/tracing/). The Working Group (WG) standardises the format and the event targets, so that logs from components written by different groups can be read side by side. That is the entire value of standardising it: a log you cannot line up against somebody else's answers no question worth asking.

The Admiralty additionally records, for the whole game: the seed, the parameters, the turn mode, and every submission it accepted with its round number. That record plus the seed reproduces the game exactly (see the **Main** document on determinism), which is what makes a finished game replayable and a disagreement settleable.

Relayed messages are logged too. Their contents prove nothing — they may be lies — but their existence and their timing can at least be established afterwards.

13 What the Working Group (WG) must settle

The **Main** document lists the decisions the Working Group (WG) owes for the game's *rules*. What follows is the same list for its *messages*, and each item is open because it is a decision rather than an oversight:

1. **The conflict order and every conflict outcome** (section 7.4, Conflicts). The most important item on this list, and the one with the largest consequences.

2. **The `Action` enumeration.** The **Main** document lists the actions and what they mean. `Move` is settled — it carries the single neighbouring `Cell` to sail to — but the rest is not: how an Island is named, and what an Island's answer to `AskClue` looks like when it is neither a Clue nor a refusal. Note that `AcceptAlliance` has to name a proposer: a Ship may hold one outgoing proposal but any number of incoming ones (see the **Main** document).

3. **The `Event` enumeration.** What a Ship is told happened, as distinct from what it can see. Take care: an event is as capable of leaking information as a view is, and it is easier to forget. What a bombardment reveals is already settled in the **Main** document — the victim learns who fired, the attacker learns whether it hit — but the shape of the events that carry it is not. A Ship striking a rock needs an event too, and so does a Ship sinking on one with nobody to inherit its Clues. **Three things do not follow line of sight**, and the **Main** document fixes all three: an alliance formed or broken, which goes to all six Ships; a declaration, which goes to all six and carries the cell that was named; and the name of whoever fires a shot, which reaches the victim and every onlooker who can see the victim's cell, even when the attacker is nowhere any of them could see. Everything else is filtered by what the Ship can see, and no fourth exception may be invented here.

4. **`ShipView` and `SpectatorView`** in detail. What may be in them is settled in the **Main** document; their shape is not.

5. **The Island trait**, likewise. Two of its methods are not optional: the **Main** document obliges an Island to be able to say that it has nothing further to offer a given Ship, and to say how much longer that Ship must wait. Both have to exist in the trait, or an Island cannot keep a promise the **Main** document makes on its behalf. The answer type for `AskClue` belongs here as well, and must be able to express all three legal outcomes: a Clue, a refusal, and "not yet, come back in n rounds". A Clue it returns is always of the same variant as the one it was given.

6. `Outcome`, carried by `GameOver` (section 7, Rounds) and returned by `Chart::outcome`. The **Main** document fixes what is ranked — how many of the world's Clues a Ship holds at full strength, ties by identifier — so what is open is the shape: whether a Ship is told the whole ordering or only its own place, and how much of either a Ship that sank in round twelve can be told, given that the game had not finished when its `GameOver` was sent. Note two things the Chart must carry for it: a sunk Ship's count, recorded as it sinks, because its Clues pass to the killer a moment later; and the fact that a game ended with nobody afloat, which no `EndReason` expresses.

7. `GenError`, returned by `generate`. The **Main** document gives it at least two causes — parameters refused outright, and rejection sampling that ran out of attempts — and a harness that cannot tell them apart cannot say whether to fix the parameters or try another seed.

8. **Deadline behaviour** when a submission arrives in the same instant it expires.

9. **Channel capacity**, and what the Admiralty does when a Ship's inbound channel is full.

One piece of advice on how to run this. The productive way to find the holes in a protocol is to try to abuse it. Take a message and ask what happens if it arrives twice, arrives late, arrives before `Welcome`, or arrives from somebody who should not be able to send it. Draw the states a Ship can be in — sailing, moored, sunk — and look for the transitions nobody has named. The problems worth finding are almost never in a single message; they are in two features that were each specified alone.